BCSE302 L - Case Study 1: Work Division Plan

Exploring Modern Database Technologies | Team of 3 | Deadline: 29-10-2026 | 10 Marks

1. Suggested Topic (change if you already have one)

Graph database: University Course Prerequisite & Recommendation Network using Neo4j, compared against a relational (MySQL/PostgreSQL) design. It is small, easy to demo, and graph queries (multi-level prerequisites, shortest path) clearly show why a graph DB beats SQL joins. Other options from the brief: fraud relationship analysis, social network, or a Firebase/MongoDB Atlas cloud app.

2. Roles at a Glance

Member	Role	Main Responsibility
Member 1	Research & Report Lead	Problem statement, technology study, advantages/limitations, comparison write-up, final report compilation, references
Member 2	Database & Prototype Lead	Neo4j setup, graph model, data loading, Cypher queries, small working demo (and basic UI if time)
Member 3	Relational Comparison & Presentation Lead	SQL schema, same queries in SQL, performance/complexity comparison, screenshots, PPT, demo rehearsal

3. Detailed Tasks

Member 1 - Research & Report Lead (rubrics 1, 2, 5)

• Write Title, Problem Statement and Objective (real-world problem: tracking course prerequisites and recommending courses).

• Study Neo4j: nodes, relationships, properties, Cypher, ACID, use cases. Write the 'Selected Technology' section.

• Explain why a graph DB suits this problem (relationship-heavy, variable-depth traversal).

• Write Advantages and Limitations, Conclusion and References.

• Merge all parts into one report, format it and upload on VTOP.

Member 2 - Database & Prototype Lead (rubrics 3, 4)

• Install Neo4j (Desktop/Aura free). Design the graph: Course, Student, Department nodes; PREREQUISITE_OF, ENROLLED_IN, BELONGS_TO relationships.

• Load sample data (20-30 courses, 10-15 students) using CSV or CREATE statements.

• Write 5-6 Cypher queries: direct prerequisites, full prerequisite chain, courses unlocked after a course, shortest learning path, recommended courses, circular dependency check.

• Optional: simple Python/Streamlit front end. Share the Cypher script and screenshots with the team.

Member 3 - Relational Comparison & Presentation Lead (rubrics 4, 5)

• Design the relational schema (Course, Student, Prerequisite, Enrollment) with ER diagram, and load the same data in MySQL/PostgreSQL.

• Write equivalent SQL for the same queries (recursive CTEs / self-joins) and compare: query complexity, readability, performance, scalability.

• Prepare the comparison table for the report (graph vs relational).

• Create the presentation slides, collect screenshots, and organise the demo flow and viva Q&A.

4. Timeline (3 days to deadline)

Day	Member 1	Member 2	Member 3
Day 1 (26 Oct)	Finalise topic with team; study Neo4j; draft problem statement and objective	Install Neo4j; design graph model; load data	Install MySQL/PostgreSQL; design ER diagram and tables; load data

Day	Member 1	Member 2	Member 3
Day 2 (27-28 Oct)	Write technology, advantages, limitations sections	Write and test Cypher queries; build demo	Write SQL queries; run comparison; start PPT
Day 3 (28-29 Oct)	Compile report, conclusion, references; proofread	Final demo run; give screenshots and scripts to team	Finish PPT; add comparison to report; rehearse demo together
Submit (29 Oct)	Upload report PDF on VTOP before deadline	Keep demo ready	Keep PPT ready

5. Report Sections Mapped to Owners

Report Section	Owner	Support
Title, Problem Statement, Objective	Member 1	-
Selected Technology	Member 1	Member 2
Proposed Solution (design + implementation)	Member 2	Member 3
Comparison with relational approach	Member 3	Member 2
Advantages and Limitations	Member 1	Member 3
Conclusion and References	Member 1	All

6. Marks Checklist

Rubric (2 marks each)	What to make sure of	Owner
Problem identification and relevance	Clear real-world problem, why it matters	Member 1
Understanding of technology	Concepts, features, why it fits	Member 1
Database design and implementation	Working graph model, data, queries	Member 2
Analysis of technical insights	Graph vs relational comparison with evidence	Member 3 + 2
Report, demo and documentation	Clean report, smooth demo, slides	Member 3 + 1

Team tips: use one shared folder (Google Drive/GitHub), agree on the data and query list on Day 1, hold a short sync each evening, and have every member able to explain the whole project during the demo.

Member 2 Guide: Neo4j Graph Design & Queries

Course Prerequisite Network | BCSE302 L Case Study 1

1. Setup (15 minutes)

• Easiest: create a free **Neo4j Aura** instance (console.neo4j.io) and open **Query**/Browser. Alternative: install Neo4j Desktop.
• Save the password shown at creation. Run the scripts below in the query box one block at a time.
• Run `MATCH (n) DETACH DELETE n;` any time to reset and start over.

2. Graph Model

Node label	Properties	Meaning
Course	code, name, credits	A subject offered by the university
Department	code, name	CSE or MATH
Student	regNo, name	A student of the university

Relationship	Direction	Meaning
PREREQUISITE_OF	(Course)->(Course)	CS101 PREREQUISITE_OF CS201 means CS101 must be done before CS201
BELONGS_TO	(Course)->(Department)	Course is offered by the department
COMPLETED {grade}	(Student)->(Course)	Student has passed the course
ENROLLED_IN	(Student)->(Course)	Student is currently taking the course

Prerequisite structure used in the sample data: CS101 -> CS201 -> CS301 -> CS402; CS201 -> CS302, CS303; MA101 -> CS202, MA201; CS201 and CS202 -> CS302; CS302 and MA201 -> CS401; CS302 -> CS403.

3. Sample Data Script (copy, paste, run once)

```
CREATE (cse:Department {code:'CSE', name:'Computer Science'}),
       (mat:Department {code:'MAT', name:'Mathematics'}),
(cs101:Course {code:'CS101', name:'Intro to Programming', credits:4}),
(ma101:Course {code:'MA101', name:'Calculus', credits:4}),
(cs201:Course {code:'CS201', name:'Data Structures', credits:4}),
(cs202:Course {code:'CS202', name:'Discrete Mathematics', credits:3}),
(ma201:Course {code:'MA201', name:'Linear Algebra', credits:3}),
(cs301:Course {code:'CS301', name:'Database Systems', credits:4}),
(cs302:Course {code:'CS302', name:'Algorithms', credits:4}),
(cs303:Course {code:'CS303', name:'Operating Systems', credits:4}),
(cs401:Course {code:'CS401', name:'Machine Learning', credits:3}),
(cs402:Course {code:'CS402', name:'Big Data Analytics', credits:3}),
(cs403:Course {code:'CS403', name:'Artificial Intelligence', credits:3}),
(cs101)-[:PREREQUISITE_OF]->(cs201),
(ma101)-[:PREREQUISITE_OF]->(cs202),
(ma101)-[:PREREQUISITE_OF]->(ma201),
(cs201)-[:PREREQUISITE_OF]->(cs301),
(cs201)-[:PREREQUISITE_OF]->(cs302),
(cs202)-[:PREREQUISITE_OF]->(cs302),
(cs201)-[:PREREQUISITE_OF]->(cs303),
(cs302)-[:PREREQUISITE_OF]->(cs401),
(ma201)-[:PREREQUISITE_OF]->(cs401),
(cs301)-[:PREREQUISITE_OF]->(cs402),
(cs302)-[:PREREQUISITE_OF]->(cs403),
(cs101)-[:BELONGS_TO]->(cse), (cs201)-[:BELONGS_TO]->(cse),
(cs202)-[:BELONGS_TO]->(cse), (cs301)-[:BELONGS_TO]->(cse),
(cs302)-[:BELONGS_TO]->(cse), (cs303)-[:BELONGS_TO]->(cse),
(cs401)-[:BELONGS_TO]->(cse), (cs402)-[:BELONGS_TO]->(cse),
(cs403)-[:BELONGS_TO]->(cse),
(ma101)-[:BELONGS_TO]->(mat), (ma201)-[:BELONGS_TO]->(mat),
(s1:Student {regNo:'S1', name:'Asha'}),
```

```
(s2:Student {regNo:'S2', name:'Ravi'}),
(s3:Student {regNo:'S3', name:'Meena'}),
(s1)-[:COMPLETED {grade:'A'}]->(cs101),
(s1)-[:COMPLETED {grade:'B'}]->(ma101),
(s1)-[:COMPLETED {grade:'A'}]->(cs201),
(s2)-[:COMPLETED {grade:'B'}]->(cs101),
(s2)-[:COMPLETED {grade:'B'}]->(ma101),
(s3)-[:COMPLETED {grade:'A'}]->(cs101),
(s3)-[:COMPLETED {grade:'A'}]->(ma101),
(s3)-[:COMPLETED {grade:'B'}]->(cs201),
(s3)-[:COMPLETED {grade:'A'}]->(cs202),
(s3)-[:COMPLETED {grade:'A'}]->(ma201),
(s3)-[:ENROLLED_IN]->(cs302);
```

Check with `MATCH (n) RETURN n;` to see the whole graph visually (take a screenshot for the report).

4. Cypher Queries for the Demo

Q1. Direct prerequisites of Database Systems

```
MATCH (p:Course)-[:PREREQUISITE_OF]->(c:Course {code:'CS301'})
RETURN p.code, p.name;
```

Q2. Full prerequisite chain of Machine Learning (any depth)

```
MATCH (p:Course)-[:PREREQUISITE_OF*1..]->(c:Course {code:'CS401'})
RETURN DISTINCT p.code, p.name;
```

Q3. All courses unlocked by Intro to Programming

```
MATCH (c:Course {code:'CS101'})-[:PREREQUISITE_OF*1..]->(n:Course)
RETURN DISTINCT n.code, n.name;
```

Q4. Shortest learning path from CS101 to CS401

```
MATCH path = shortestPath(
  (a:Course {code:'CS101'})-[:PREREQUISITE_OF*]->(b:Course {code:'CS401'}))
RETURN [n IN nodes(path) | n.code] AS route;
```

Q5. Courses a student is eligible for (all prerequisites completed)

```
MATCH (s:Student {regNo:'S1'}), (c:Course)
WHERE NOT (s)-[:COMPLETED]->(c) AND NOT (s)-[:ENROLLED_IN]->(c)
  AND ALL(x IN [(q:Course)-[:PREREQUISITE_OF]->(c) | q]
          WHERE (s)-[:COMPLETED]->(x))
RETURN c.code, c.name;
```

Q6. Most foundational courses (unlock the most courses)

```
MATCH (c:Course)-[:PREREQUISITE_OF*1..]->(n:Course)
RETURN c.code, c.name, count(DISTINCT n) AS unlocks
ORDER BY unlocks DESC LIMIT 3;
```

Q7. Detect circular prerequisites (data validation)

```
MATCH (c:Course)-[:PREREQUISITE_OF*1..]->(c)
RETURN c.code;
```

Q8. Number of courses per department

```
MATCH (c:Course)-[:BELONGS_TO]->(d:Department)
RETURN d.name, count(c) AS courses;
```

Expected results to check: Q1 gives CS201. Q2 gives CS302, CS201, CS101, CS202, MA101, MA201. Q5 for S1 (Asha) gives CS202, MA201, CS301 and CS303 (CS302 is excluded because Asha has not completed CS202). Q7 returns nothing (no cycles).

5. Relational Equivalent (hand over to Member 3)

```
CREATE TABLE course (code VARCHAR(10) PRIMARY KEY, name VARCHAR(60), credits INT);
CREATE TABLE prerequisite (
  prereq_code VARCHAR(10) REFERENCES course(code),
  course_code VARCHAR(10) REFERENCES course(code),
  PRIMARY KEY (prereq_code, course_code));

-- Q2 in SQL: needs a recursive CTE (PostgreSQL)
WITH RECURSIVE chain(prereq_code, course_code) AS (
  SELECT prereq_code, course_code FROM prerequisite
  WHERE course_code = 'CS401'
  UNION
  SELECT p.prereq_code, p.course_code
  FROM prerequisite p JOIN chain c ON p.course_code = c.prereq_code)
SELECT DISTINCT c.code, c.name
FROM chain JOIN course c ON c.code = chain.prereq_code;
```

Point for the report: the Cypher version is a single line of pattern matching, while SQL needs a recursive CTE and extra joins. Depth in Cypher is just `*1..`.

6. Screenshots to Collect

• Whole graph view (`MATCH (n) RETURN n`) and the Course/Department sub-graph.
• Output of Q2, Q4 and Q5 in graph view and in table view.

• The SQL recursive query result from Member 3 for side-by-side comparison.
• Optional: a small Streamlit or Python script that takes a student ID and prints eligible courses (neo4j Python driver).

7. Demo Flow (about 5 minutes)

• Show the problem and the graph (all nodes and relationships).
• Run Q2, then show the same in SQL (recursive CTE) to compare effort.
• Run Q4 (shortest path) and Q5 (eligible courses for a student).
• Add a new course and prerequisite live with CREATE/MATCH, then re-run Q3 to show how flexible the model is.
Live add example:

```
MATCH (a:Course {code:'CS301'})
CREATE (n:Course {code:'CS404', name:'Data Mining', credits:3})
CREATE (a)-[:PREREQUISITE_OF]->(n);
```